jvo, hdaniel, jebarateiro

Lab 5: Navegação

Este lab deve ser feito individualmente.

Submeta o seu código ao mooshak http://deei-mooshak.ualg.pt/~jvo/ usando o seu login da ualg (sem @ualg.pt). Ex: a123456

Este lab é composto por dois problemas.

Uma submissão ao problema permanecerá *pending* até que seja validada pelo professor durante a aula prática. Só as submissões *final* serão consideradas para avaliação.

Todas as submissões deverão ser feitas até

6 de abril 2026

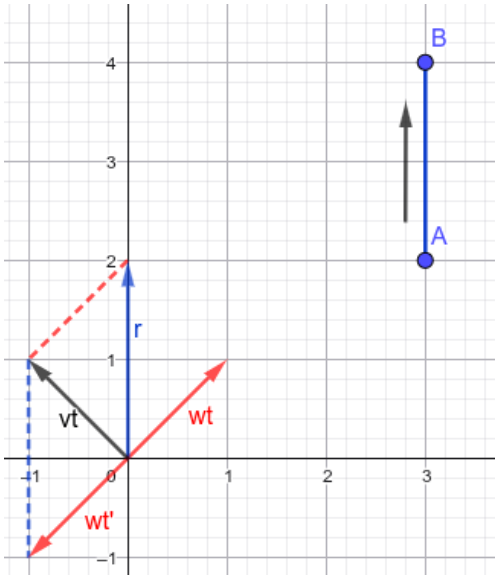
A validação poderá ser feita posteriormente, se necessário, até 16 de abril de 2026

NB: Nos problemas seguintes, sempre que necessário, considera-se que dois double *d* e *g* são iguais se |*d*-*g*|<1e-9

Reveja agora os slides das teóricas 3 e seguintes antes de responder aos problemas abaixo

Problema K: Velocidade e tempo

Pretende-se calcular uma velocidade constante $\vec{v}$ para deslocar um avião do ponto **A** para o ponto **B** num dado tempo ***t***, mantendo a altitude constante e sabendo que o vento tem velocidade constante $\vec{w}$.



$\overrightarrow{wt}' = -\overrightarrow{wt}$

Sendo as velocidades grandezas vetoriais em R^2, criando um vetor posição $\vec{r}$ com módulo igual à distância de **A** a **B** e direção de **A** para **B**, subtraindo a velocidade do vento $\vec{w}$ e dividindo pelo tempo de viagem desejado ***t***, obtém-se a velocidade do avião $\vec{v}$:

$$\vec{v} = \frac{\vec{r} + \overrightarrow{wt}'}{t} = \frac{\vec{r} - \overrightarrow{wt}}{t} = \frac{\vec{r}}{t} - \vec{w}$$

Tendo o avião uma velocidade linear constante ***vl***, o tempo para percorrer a distância ***r*** em linha reta do ponto **A** ao ponto **B** é dado por:

$$t = \frac{r}{vl}$$

Reveja as funções definidas no problema H na classe **AutoPilot** de modo que cumpram o descrito abaixo:

Vetor speed(Vetor w, double t)

retorna a velocidade vetorial $\vec{v}$ necessária para que um avião se desloque em linha reta do ponto A ao B, pontos estes definidos no construtor, com vento lateral com velocidade vetorial $\vec{w}$, no tempo ***t***.

double time(double vl)

retorna o tempo necessário ***t*** para que um avião se desloque em linha reta do ponto A ao B, pontos estes definidos no construtor, com a velocidade linear ***vl***.

Reveja os testes de unidade para **estes 2 métodos**.

A função **main()** da classe **Cliente** deve ser a indicada abaixo:

```
public static void main() {
    Scanner sc = new Scanner(System.in);

    //Get start and finish points
    Point start = new Point(sc.nextDouble(), sc.nextDouble());
    Point finish = new Point(sc.nextDouble(), sc.nextDouble());

    //Get wind speed and direction
    Vetor w = new Vetor(sc.nextDouble(), sc.nextDouble());

    //Get linear speed
    double s = sc.nextDouble();
    sc.close();

    //Setup auto pilot and compute:
    // i)  desired time to reach the finish point
    // ii) vectorial speed required
    AutoPilot ap = new AutoPilot(start, finish);
    double t = ap.time(s);
    IO.println(String.format("%.2f", t));
    IO.println(ap.speed(w, t));
}
```

Entrada

A entrada é composta por 3 linhas.

A primeira linha contém 4 doubles, x_A y_A x_B y_B, separados por espaços, que representam as coordenadas cartesianas dos pontos A e B, respetivamente, o início e o fim da trajetória.

A segunda linha contém dois doubles, x_w **e** y_w, separados por espaços, que representam as duas componentes do vetor que indica a velocidade do vento em cada eixo X e Y multiplicado pelo tempo: **wt**. A terceira linha contém um double ***vl*** com a velocidade linear desejada para efetuar a viagem de **A** a **B**.

Saída

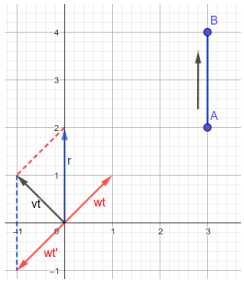
A saída é composta por 2 linhas.

A primeira linha com o tempo necessário para efetuar a viagem, arredondado a duas casas decimais.

A segunda linha com o vetor velocidade do avião $\vec{v}$, no formato [x,y] arredondado a duas casas decimais.

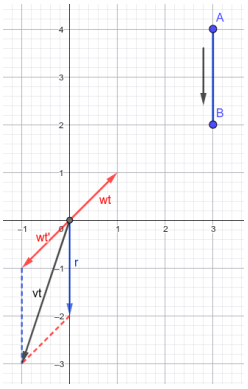
Exemplo de Entrada 1
3 2 3 4
0.2 0.2
0.4

Exemplo de Saída 1
5.00
[-0.20,0.20]



Exemplo de Entrada 2
3 4 3 2
0.2 0.2
0.4

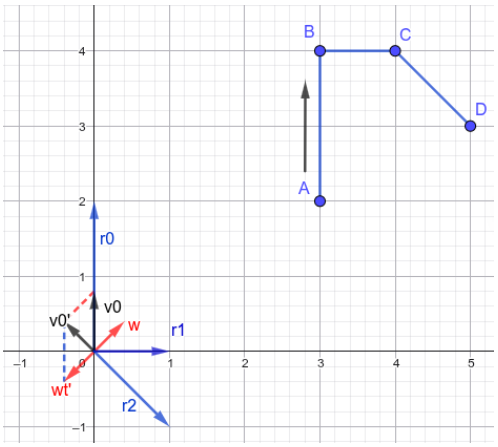
Exemplo de Saída 2
5.00
[-0.20,-0.60]



Problema L: Navegação

Pretende-se estender as funcionalidades de navegação para rotas compostas de segmentos de reta. O exemplo da figura apresenta uma rota que se inicia no ponto **A** e termina no ponto **B**. A navegação poderá ser efetuada segmento a segmento, de acordo com os princípios usados no problema anterior.

Na figura temos a decomposição dos 3 segmentos da rota: AB, BC e CD em 3 vetores r0, r1 e r3. Temos também o exemplo do cálculo da velocidade ***v0'*** para compensar a velocidade do vento ***w***, resultando na velocidade ***v0*** que mantém o avião na rota, para o primeiro segmento da rota. O cálculo para os segmentos seguintes é efetuado da mesma forma.



Este programa deve ter como base as classes implementadas nos Labs 1,2,3 e 4 e o problema K. Se não tiver, independentemente do mérito da implementação, terá avaliação zero.

Restrições complementares

Nota: Submissões não aceites têm avaliação zero. O não cumprimento de cada uma das restrições abaixo para uma submissão Accepted implica uma redução de 25% na nota da submissão. No limite a nota será zero se todas as restrições não forem cumpridas mesmo que a submissão tenha sido aceite pelo Mooshak.

1) Seguindo os princípios da programação orientada por objetos, o programa deverá estar estruturado segundo um conjunto de classes apropriadas, não duplicando código e estendendo o problema K.

2) Criar testes de unidade para **todos os métodos de todas as classes**.

3) Todas as variáveis de instância das classes têm de ter visibilidade private ou protected, sendo acedidas apenas por acessores de leitura. Não podem ter assessores de escrita.
A classe **Cliente** apenas lê os dados de entrada, processa-os chamando apenas os métodos necessários implementados nas classes e escreve o resultado na consola.

4) Nos comentários indique:
   i) Uma linha com a descrição da responsabilidade da classe ou do que o método faz
   ii) @author (apenas para classes)

iii) @version (apenas para classes; inclua uma data)

iv) @inv (apenas para classes; inclua uma descrição da invariante usada)

v) @param (apenas para métodos e construtores)

vi) @return (apenas para métodos)

vii) @see (qualquer referência bibliográfica ou sítio web consultado para o desenvolvimento do código respetivo)

viii) Pré e pós-condições para cada método quando necessário

Entrada

A entrada é composta por 4 linhas.

A primeira linha contém um número par de doubles separados por espaços. Cada par representa as coordenadas cartesianas x e y de cada ponto da rota.

A segunda linha contém dois doubles, x_w **e** y_w, separados por espaços, que representam as duas componentes do vetor que indica a velocidade do vento em cada eixo X e Y: w.

A terceira linha contém um double vl com a velocidade linear constante desejada para percorrer toda a rota.

A quarta linha contém um double t com um tempo desde o início da viagem, para o qual queremos calcular a posição.

Saída

Se a rota fornecida tiver um número ímpar de coordenadas, apenas uma linha com o texto: "Rota:iv".

Se a rota fornecida tiver um número par de coordenadas 4 linhas, como descrito abaixo:

A primeira linha com o comprimento da rota, arredondado a duas casas decimais.

A segunda linha com o tempo necessário para efetuar a viagem, arredondado a duas casas decimais.

A terceira linha com a posição do avião no tempo t após ter iniciado a viagem.

A quarta linha com o uma lista de vetores velocidade do avião $\vec{v}$, no formato [x,y] arredondado a duas casas decimais, um para cada segmento da rota.

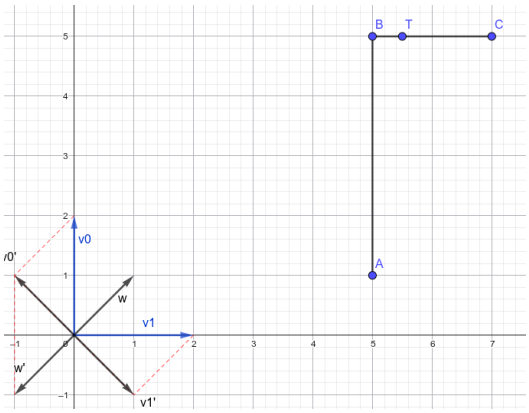
Exemplo de Entrada 1

5 2 3
2 3
0.5
2

Exemplo de Saída 1

Rota:iv

Exemplo de Entrada 2
5 1 5 5 7 5
1 1
2
2.25



Exemplo de Saída 2
6.00
3.00
(5.50,5.00)
[-1.00,1.00] [1.00,-1.00]